

# Penguins – Análisis Completo

Equipo 7

Integrantes:

Fajardo Zavala Yendali Viridiana  
González Rivas José Rafael  
Maldonado Zamora Silvia Viridiana  
Olivera Flores Jonathan  
Valencia Quiroz Luis Roberto

2026-01-04

## Carga y preparación de datos

```
df <- read.csv("C:/Users/usuario/OneDrive/Desktop/Proyecto 2 Diplomado/Penguins/Club_Penguin/penguins.csv")
df %>% dim()
```

```
## [1] 344 8
```

```
df1 <- df[,1:8]
colSums(is.na(df1))
```

```
##           species           island  bill_length_mm  bill_depth_mm
##              0              0             2             2
## flipper_length_mm  body_mass_g         sex          year
##              2              2             11             0
```

```
df1c <- df1 %>% mutate(across(where(is.numeric), ~ ifelse(is.na(.), mean(., na.rm = TRUE), .)))
colSums(is.na(df1c))
```

```
##           species           island  bill_length_mm  bill_depth_mm
##              0              0             0             0
## flipper_length_mm  body_mass_g         sex          year
##              0              0             11             0
```

```
df1c <- df1c %>% mutate(
  species = as.factor(species),
  island = as.factor(island),
  sex = as.factor(sex)
)
```

## Exploración

```
df1c %>% glimpse()
```

```
## Rows: 344
## Columns: 8
## $ species      <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adel~
## $ island       <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgerse~
## $ bill_length_mm <dbl> 39.10000, 39.50000, 40.30000, 43.92193, 36.70000, 39~
## $ bill_depth_mm <dbl> 18.70000, 17.40000, 18.00000, 17.15117, 19.30000, 20~
## $ flipper_length_mm <dbl> 181.0000, 186.0000, 195.0000, 200.9152, 193.0000, 19~
## $ body_mass_g   <dbl> 3750.000, 3800.000, 3250.000, 4201.754, 3450.000, 36~
## $ sex          <fct> male, female, female, NA, female, male, female, male~
## $ year         <int> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007~
```

```
df1c %>% count(species)
```

```
##      species      n
## 1      Adelie    152
## 2 Chinstrap     68
## 3      Gentoo   124
```

```
df1c %>% count(island)
```

```
##      island      n
## 1      Biscoe   168
## 2      Dream   124
## 3 Torgersen    52
```

```
df1c %>% count(sex)
```

```
##      sex      n
## 1 female   165
## 2  male    168
## 3   <NA>    11
```

## PCA

## T-SNE

Utilizamos T-SNE como otro algoritmo de reducción de dimensionalidad, en el caso de T-SNE recordamos que opera

convirtiendo distancias en probabilidades.

## t-SNE Básico

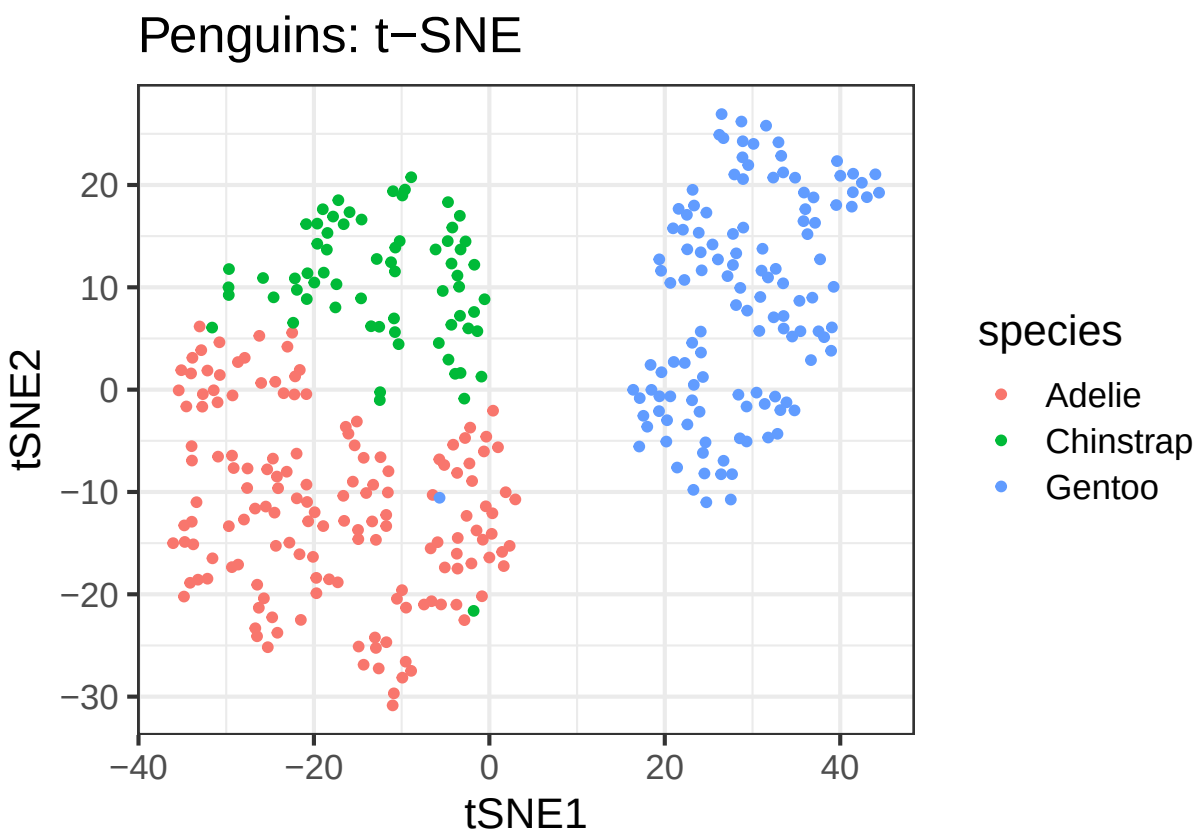
```

set.seed(564)
start.time <- proc.time()
dfpeng_tsne <- df1c %>% select(where(is.numeric)) %>% scale() %>% tsne()
proc.time() - start.time

datos <- data.frame(tSNE1 = dfpeng_tsne[,1], tSNE2 = dfpeng_tsne[,2], species = df$species)

ggplot(datos, aes(x = tSNE1, y = tSNE2, color = species)) +
  geom_point(size = 1.1) +
  labs(title = "Penguins: t-SNE")

```



## Rtsne

Para el caso de r también se tiene una versión de T-SNE particular más optimizada.

```

set.seed(564)
start.time <- proc.time()
df_Rtsne <- df1c %>% select(where(is.numeric)) %>% scale() %>% Rtsne()
proc.time() - start.time

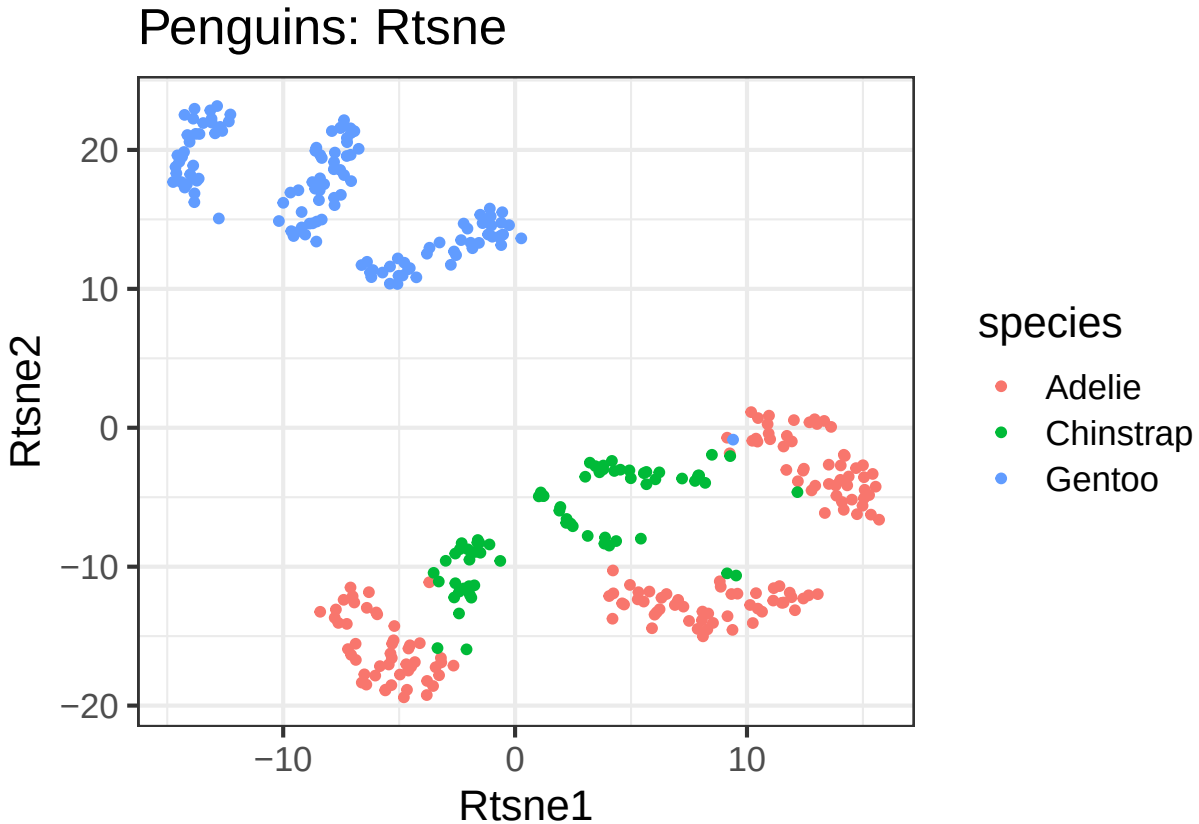
```

```

datos1 <- data.frame(Rtsne1 = df_Rtsne$Y[,1], Rtsne2 = df_Rtsne$Y[,2], species = df$species)

ggplot(datos1, aes(x = Rtsne1, y = Rtsne2, color = species)) +
  geom_point(size = 1.1) + labs(title = "Penguins: Rtsne")

```



#T-SNE 3D

```

set.seed(564)

df_tsne3 <- df1c %>%
  dplyr::select(where(is.numeric)) %>%
  scale() %>%
  tsne(k = 3)

tsne_df3 <- data.frame(
  tSNE1 = df_tsne3[, 1],
  tSNE2 = df_tsne3[, 2],
  tSNE3 = df_tsne3[, 3],
  species = factor(df$species)
)

colors <- c("#F58231", "#911EB4", "#1eb43a")

hover_text <- paste(
  "Especie:", tsne_df3$species, "",
  "Dimension 1:", round(tsne_df3$tSNE1, 3),

```

```

    "Dimension 2:", round(tsne_df3$tSNE2, 3),
    "Dimension 3:", round(tsne_df3$tSNE3, 3)
  )

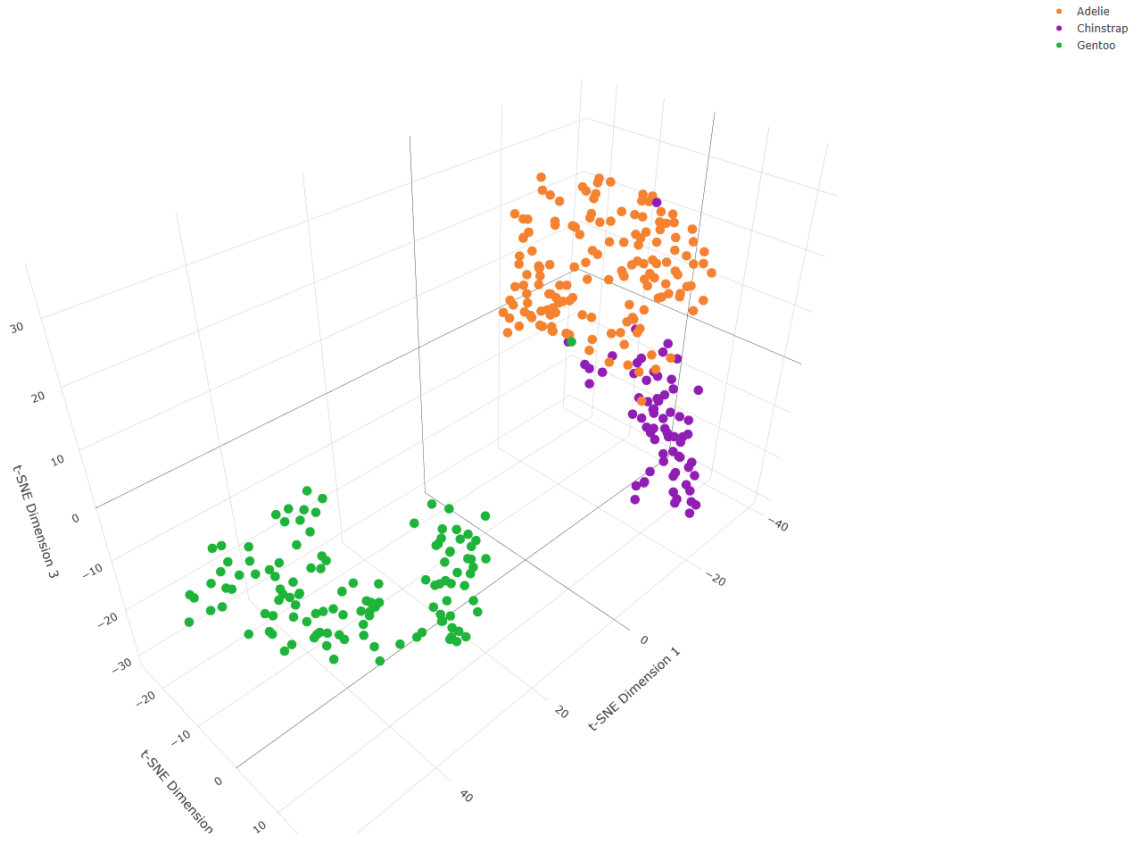
p <- plot_ly(
  data = tsne_df3,
  x = ~tSNE1,
  y = ~tSNE2,
  z = ~tSNE3,
  type = "scatter3d",
  mode = "markers",
  marker = list(size = 6),
  text = hover_text,
  hoverinfo = "text",
  color = ~species,
  colors = colors
) %>%
  layout(
    title = "t-SNE 3D: Penguins",
    scene = list(
      xaxis = list(title = "t-SNE Dimension 1"),
      yaxis = list(title = "t-SNE Dimension 2"),
      zaxis = list(title = "t-SNE Dimension 3")
    )
  )

htmlwidgets::saveWidget(p, "tsne3d_temp.html", selfcontained = TRUE)

webshot2::webshot(
  "tsne3d_temp.html",
  file = "tsne3d_plot.png",
  vwidth = 1400, vheight = 1000
)

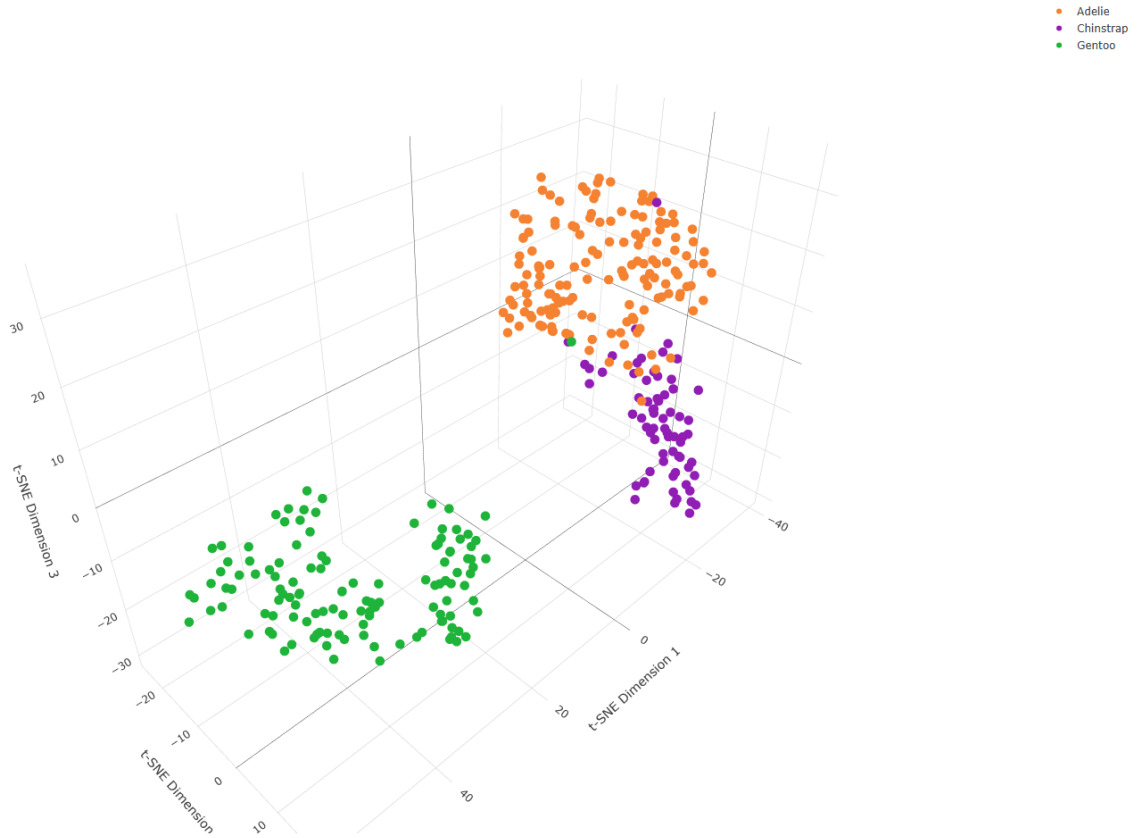
```

t-SNE 3D: Penguins



```
knitr::include_graphics("tsne3d_plot.png")
```

t-SNE 3D: Penguins



#R T-SNE 3D

```
set.seed(564)

df_Rtsne3 <- df1c %>%
  dplyr::select(where(is.numeric)) %>%
  scale() %>%
  Rtsne(dims = 3)

tsne_Rdf3 <- data.frame(
  tSNE1 = df_Rtsne3$Y[, 1],
  tSNE2 = df_Rtsne3$Y[, 2],
  tSNE3 = df_Rtsne3$Y[, 3],
  species = factor(df$species)
)

colors <- c("#F58231", "#911EB4", "#1eb43a")

hover_text <- paste(
  "Especie:", tsne_Rdf3$species, "",
  "Dimension 1:", round(tsne_Rdf3$tSNE1, 3),
  "Dimension 2:", round(tsne_Rdf3$tSNE2, 3),
  "Dimension 3:", round(tsne_Rdf3$tSNE3, 3)
)
```

```

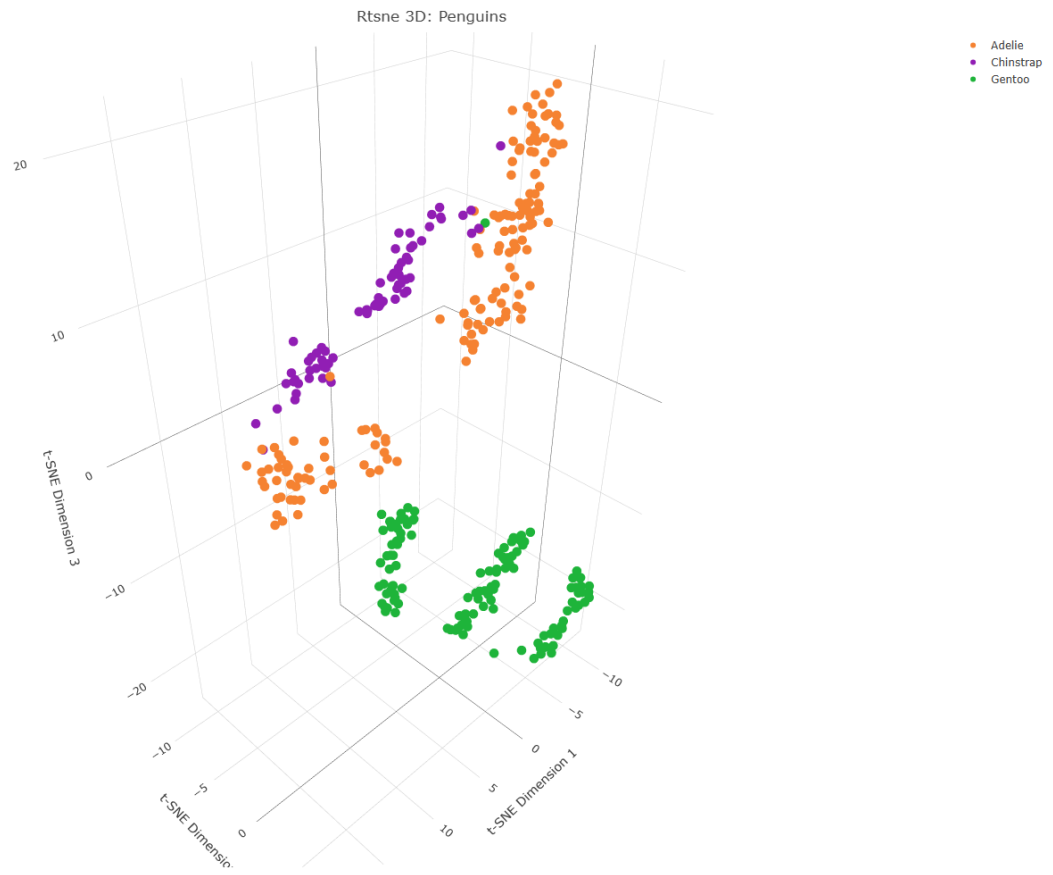
p <- plot_ly(
  data = tsne_Rdf3,
  x = ~tsNE1,
  y = ~tsNE2,
  z = ~tsNE3,
  type = "scatter3d",
  mode = "markers",
  marker = list(size = 6),
  text = hover_text,
  hoverinfo = "text",
  color = ~species,
  colors = colors
) %>%
  layout(
    title = "Rtsne 3D: Penguins",
    scene = list(
      xaxis = list(title = "t-SNE Dimension 1"),
      yaxis = list(title = "t-SNE Dimension 2"),
      zaxis = list(title = "t-SNE Dimension 3")
    )
  )

htmlwidgets::saveWidget(p, "rtsne3d_temp.html", selfcontained = TRUE)

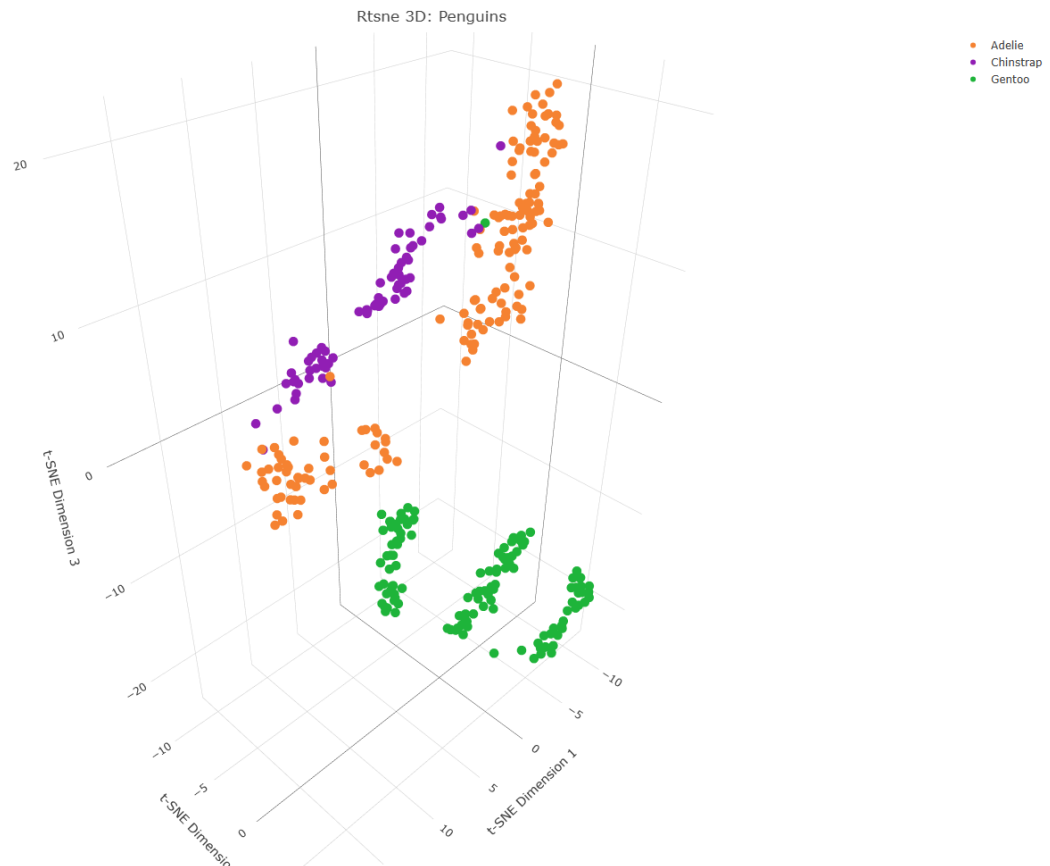
webshot2::webshot(
  "rtsne3d_temp.html",
  file = "rtsne3d_plot.png",
  vwidth = 1400, vheight = 1000
)

```





```
# ---- Insert screenshot into PDF ----  
knitr::include_graphics("rtsne3d_plot.png")
```



## Exploración de parámetros t-SNE

Aquí podemos ver que se pueden modificar de manera particular los parámetros de T-SNE para poder obtener resultados más particulares y con variaciones determinadas.

```
tsne_params <- expand.grid(perplexity = c(10,15,20,25,30,50))

numeric_df <- df1c %>% select(where(is.numeric))

set.seed(564)
tsne_res <- lapply(seq(nrow(tsne_params)), function(i) {
  tsne::tsne(X = numeric_df, max_iter = 500, perplexity = tsne_params[[1]][i])
})
```

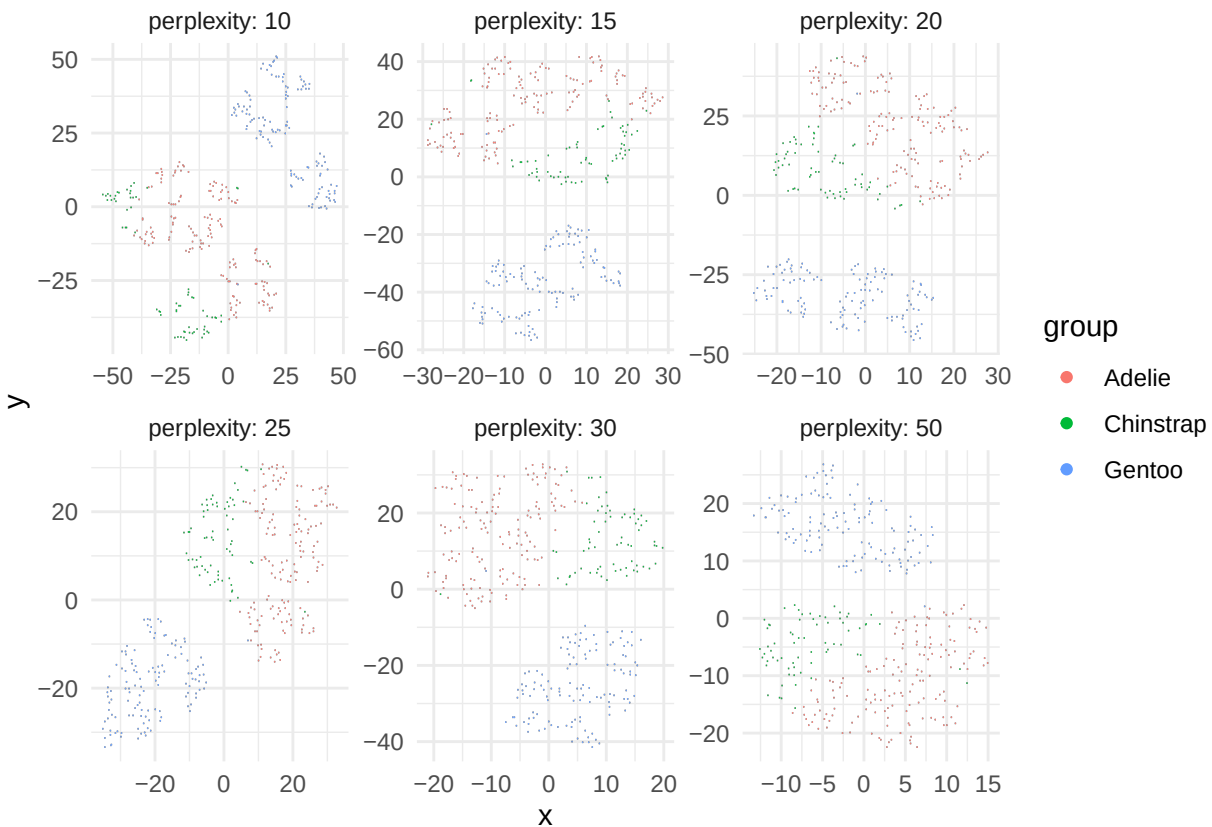
```
d1 <- rbindlist(lapply(seq(nrow(tsne_params)), function(i) {
  data.table(
    x = tsne_res[[i]][,1],
    y = tsne_res[[i]][,2],
    perplexity = tsne_params[[1]][i],
  )
}))
```

```

    group = df$species
  )
}))

ggplot(d1) +
  geom_scattermore(aes(x = x, y = y, colour = group), pointsize = 2) +
  facet_wrap("perplexity", labeller = label_both, scales = "free") +
  theme_minimal()

```



## Exploración Rtsne (perplexity y eta)

Lo mismo se puede hacer para R T-SNE.

```

set.seed(564)
Rtsne_params3 <- expand.grid(perplexity=c(10,15,20,30), eta=c(10,50,100,150,200))

Rtsne_res3 <- lapply(seq(nrow(Rtsne_params3)), function(i) {
  Rtsne(
    X = numeric_df,
    max_iter = 500,
    verbose = FALSE,
    perplexity = Rtsne_params3$perplexity[i],

```

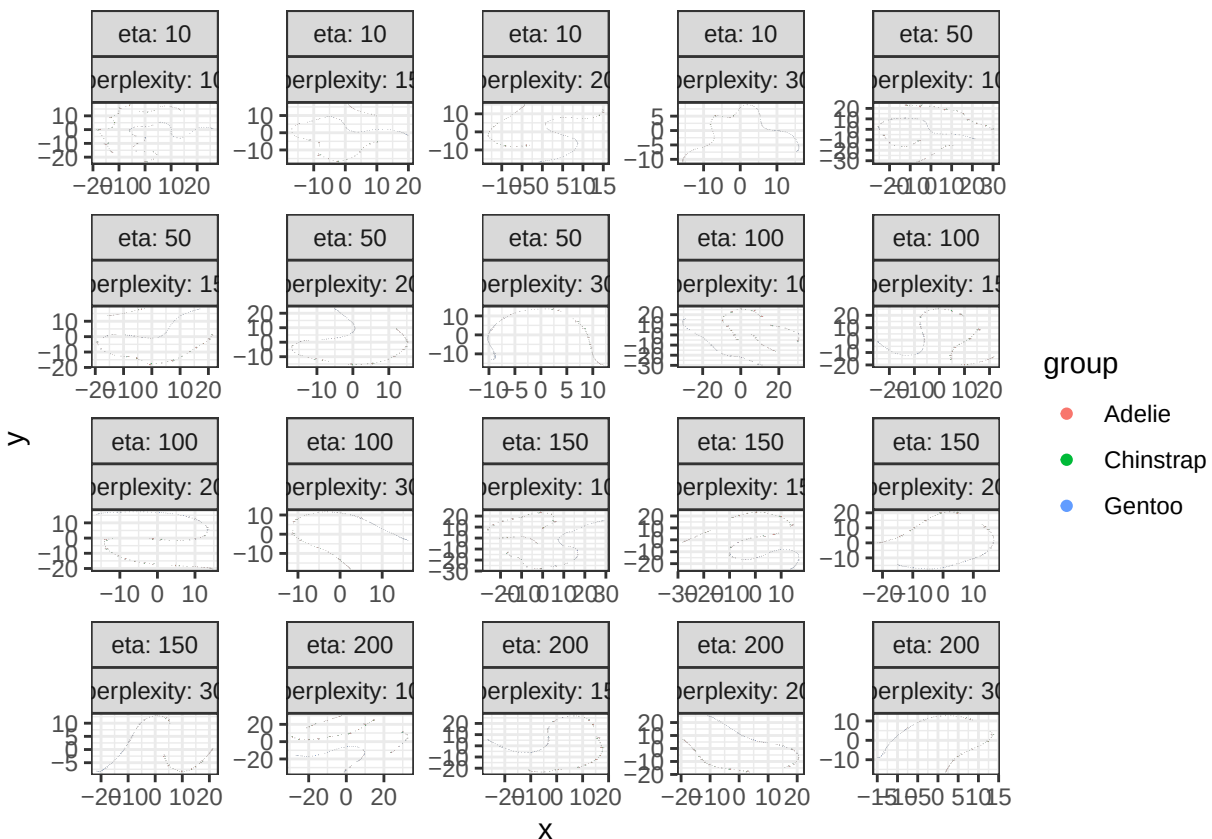
```

    eta = Rtsne_params3$eta[i],
    check_duplicates = FALSE,
    pca = FALSE
  )
})

d4 <- rbindlist(lapply(seq(nrow(Rtsne_params3)), function(i) {
  data.table(
    x = Rtsne_res3[[i]]$Y[,1],
    y = Rtsne_res3[[i]]$Y[,2],
    perplexity = Rtsne_params3[[1]][i],
    eta = Rtsne_params3[[2]][i],
    group = df$species
  )
}))

ggplot(d4) +
  geom_scattermore(aes(x = x, y = y, colour = group), pointsize = 2) +
  facet_wrap(eta~perplexity, labeller = label_both, scales = "free", ncol = 5) +
  theme_bw()

```



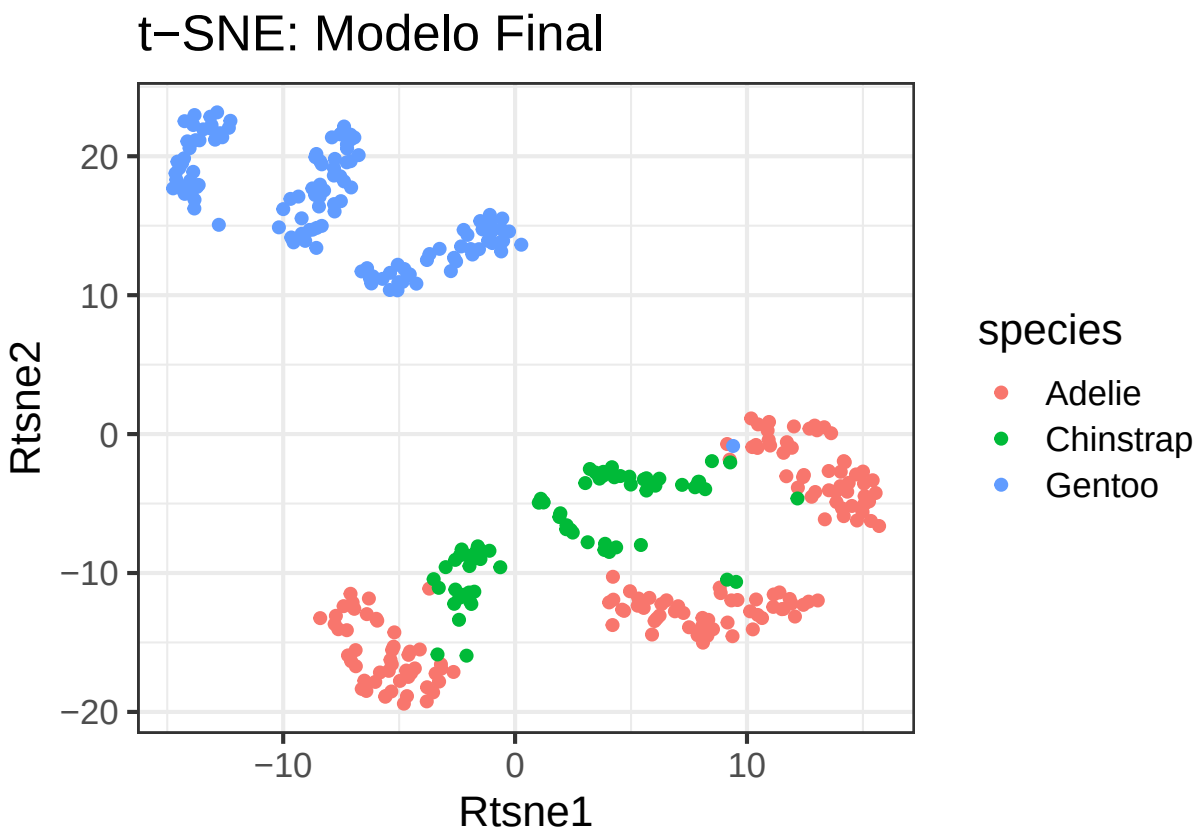
## Modelo Final

Finalmente podemos agarrar la variante con los parámetros que nos parezca que hayan dado el mejor resultado y analizarlos

en particular.

```
datos2 <- data.frame(Rtsne1 = df_Rtsne$Y[,1], Rtsne2 = df_Rtsne$Y[,2], species = df$species)

ggplot(datos2, aes(x = Rtsne1, y = Rtsne2, color = species)) +
  geom_point(size = 1.5) +
  labs(title = "t-SNE: Modelo Final")
```



## Conclusión

Podemos ver como T-SNE hace un excelente trabajo en separación de grupos, para nuestro caso particular 2 de los

grupos estaban particularmente cerca, pero en T-SNE la distancia entre grupos no tiene significado así que puede ser ignorada.

## UMAP

UMAP es nuestra tercera herramienta de reducción de dimensionalidad, aunque similar a T-SNE UMAP tiene varias ventajas sobre T-SNE

mayor velocidad y una mejor conservación de la estructura global de los datos.

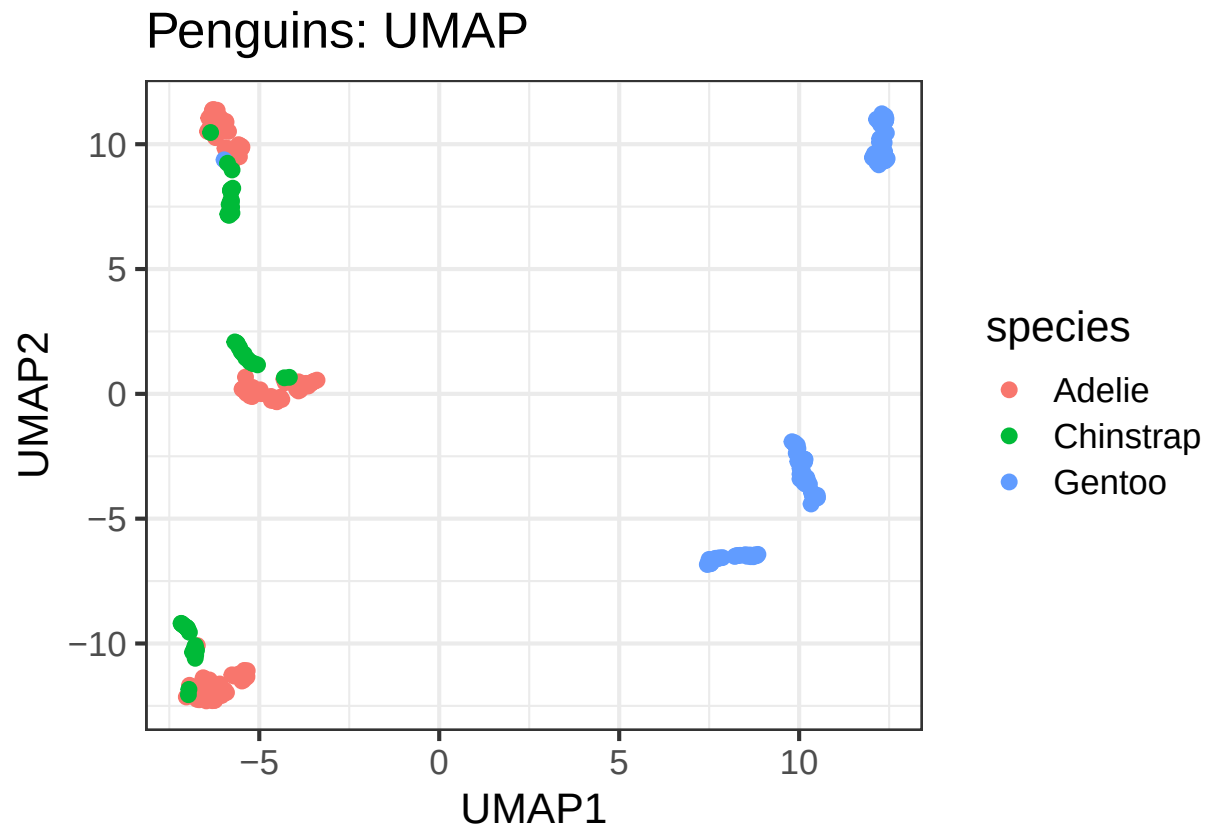
## UMAP Básico

```
df_umap <- df1c %>%
  select(where(is.numeric)) %>%
  scale() %>%
  umap()

str(df_umap)

datos <- data.frame(UMAP1=df_umap[,1], UMAP2=df_umap[,2], species=df$species)
head(datos)

ggplot(datos, aes(x = UMAP1, y = UMAP2, color = species)) +
  geom_point(size = 2) +
  labs(title="Penguins: UMAP")
```



## UMAP via tidymodels

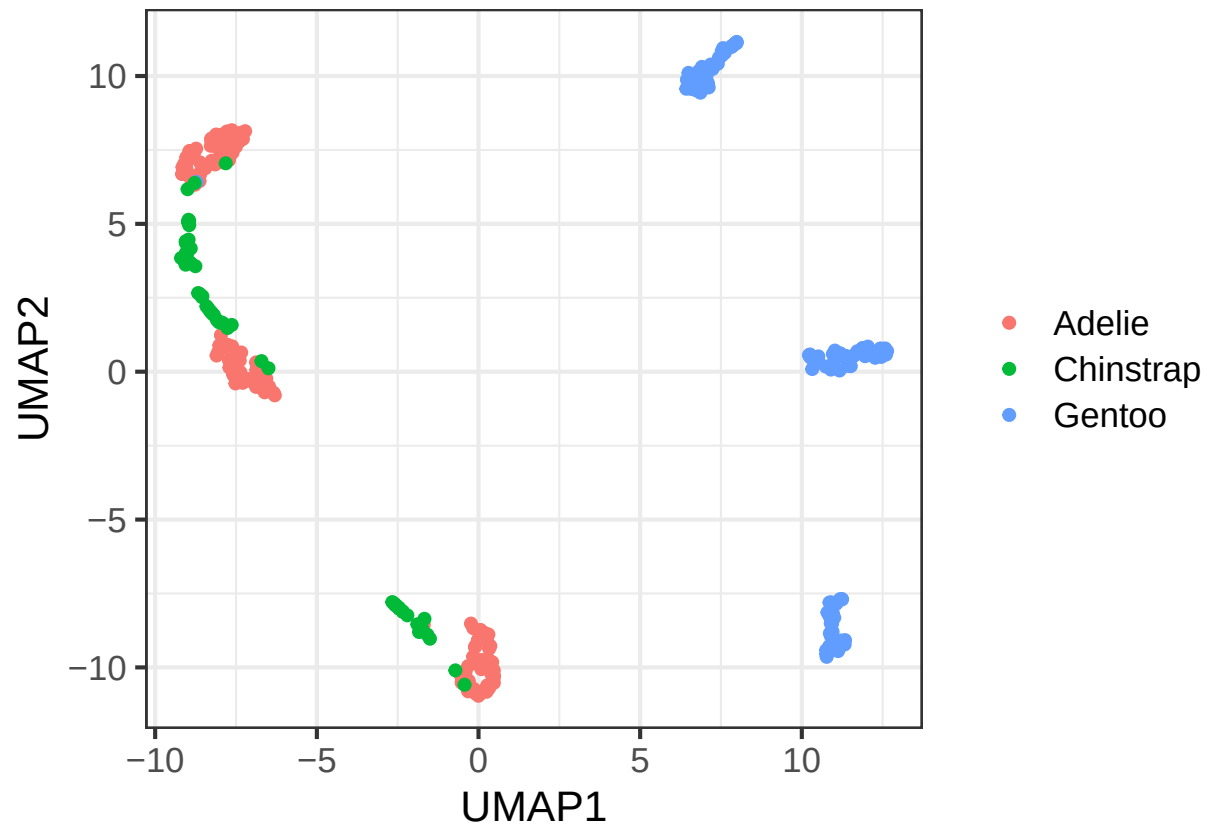
Con la libreria `tidymodels` de `r` podemos hacer UMAP tambien.

```
df1c <- df1c %>% select(-sex, -island)

umap_rec <- recipe(~., data=df1c) %>%
  update_role(species, new_role="id") %>%
  step_normalize(all_predictors()) %>%
  step_umap(all_predictors())

umap_res <- prep(umap_rec)
umap_res

juice(umap_res) %>%
  ggplot(aes(UMAP1, UMAP2)) +
  geom_point(aes(color=species), size=1.5) +
  labs(color=NULL)
```



## 3D UMAP

```
df_umap3 <- df1c %>%
  select(where(is.numeric)) %>%
  scale() %>%
  umap(n_components = 3)

umap_df3 <- data.frame(
  UMAP1 = df_umap3[,1],
  UMAP2 = df_umap3[,2],
  UMAP3 = df_umap3[,3],
  species = factor(df$species)
)

colors <- c("#F58231", "#911EB4", "#3fb41e")

hover_text <- paste(
  "species:", umap_df3$species,
  "Dimension 1:", round(umap_df3$UMAP1, 3),
  "Dimension 2:", round(umap_df3$UMAP2, 3),
  "Dimension 3:", round(umap_df3$UMAP3, 3)
)
```



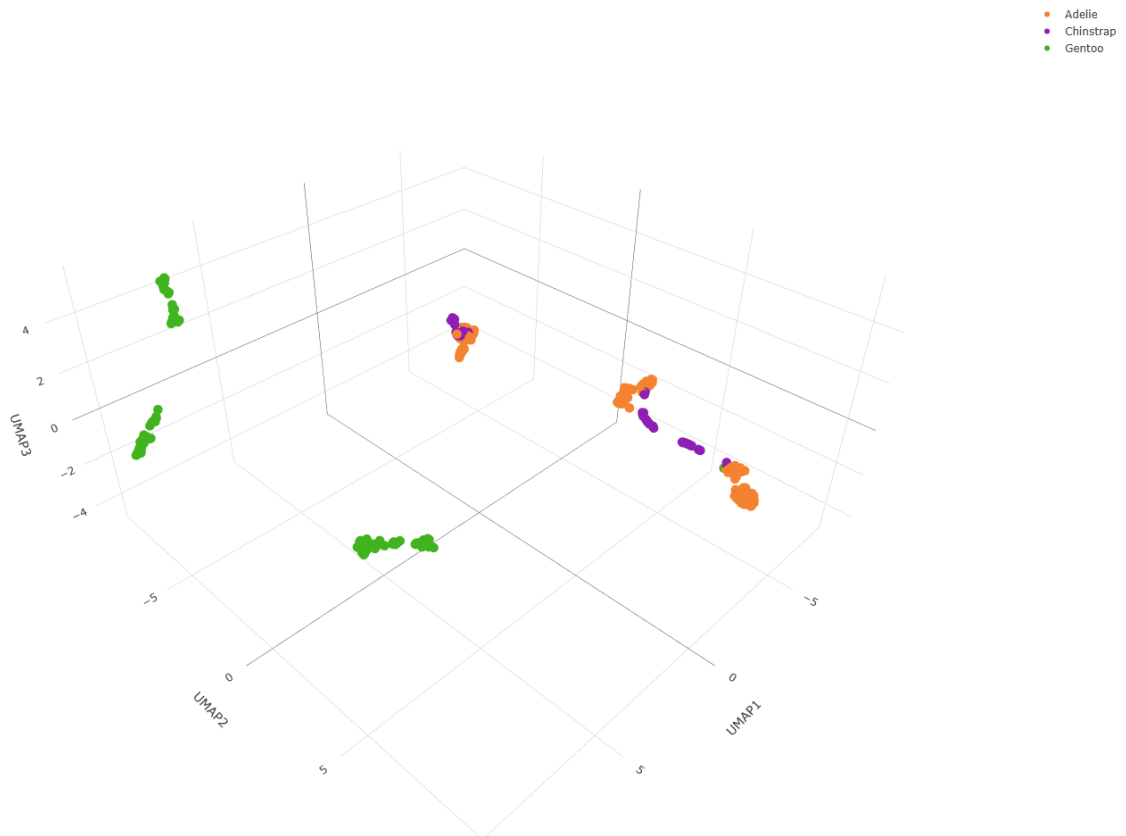
```

p <- plot_ly(
  data = umap_df3,
  x = ~UMAP1, y = ~UMAP2, z = ~UMAP3,
  type = "scatter3d", mode = "markers",
  marker = list(size = 6),
  text = hover_text, hoverinfo = "text",
  color = ~species, colors = colors
)

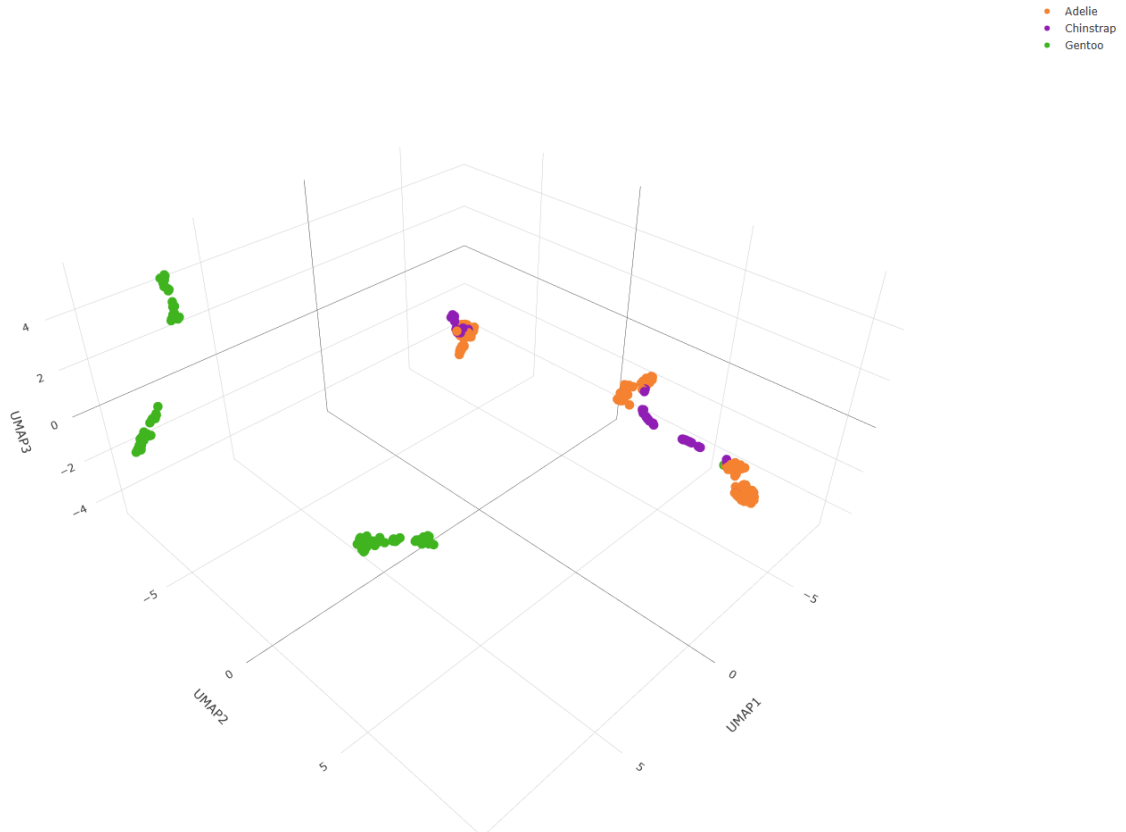
htmlwidgets::saveWidget(p, "umap3d_temp.html", selfcontained = TRUE)

webshot2::webshot(
  "umap3d_temp.html",
  file = "umap3d_plot.png",
  vwidth = 1400, vheight = 1000
)

```



```
knitr::include_graphics("umap3d_plot.png")
```



## Exploración de Hipereparámetros

De la misma manera que en T-SNE se pueden modificar de manera fina los parámetros del proceso para obtener resultados muy particulares.

```
umap_params <- expand.grid(n_neighbors=c(10,20,50,100),
                          min_dist=c(0.5,0.75,1.1))

numeric_df <- df1c %>% select(where(is.numeric))

umaps <- lapply(seq(nrow(umap_params)), function(i) {
  umap(X=numeric_df,
       n_neighbors=umap_params$n_neighbors[i],
       min_dist=umap_params$min_dist[i])
})

d <- rbindlist(lapply(seq(nrow(umap_params)), function(i) {
  data.table(
    x=umaps[[i]][,1], y=umaps[[i]][,2],
    n_neighbors=umap_params$n_neighbors[i],
```

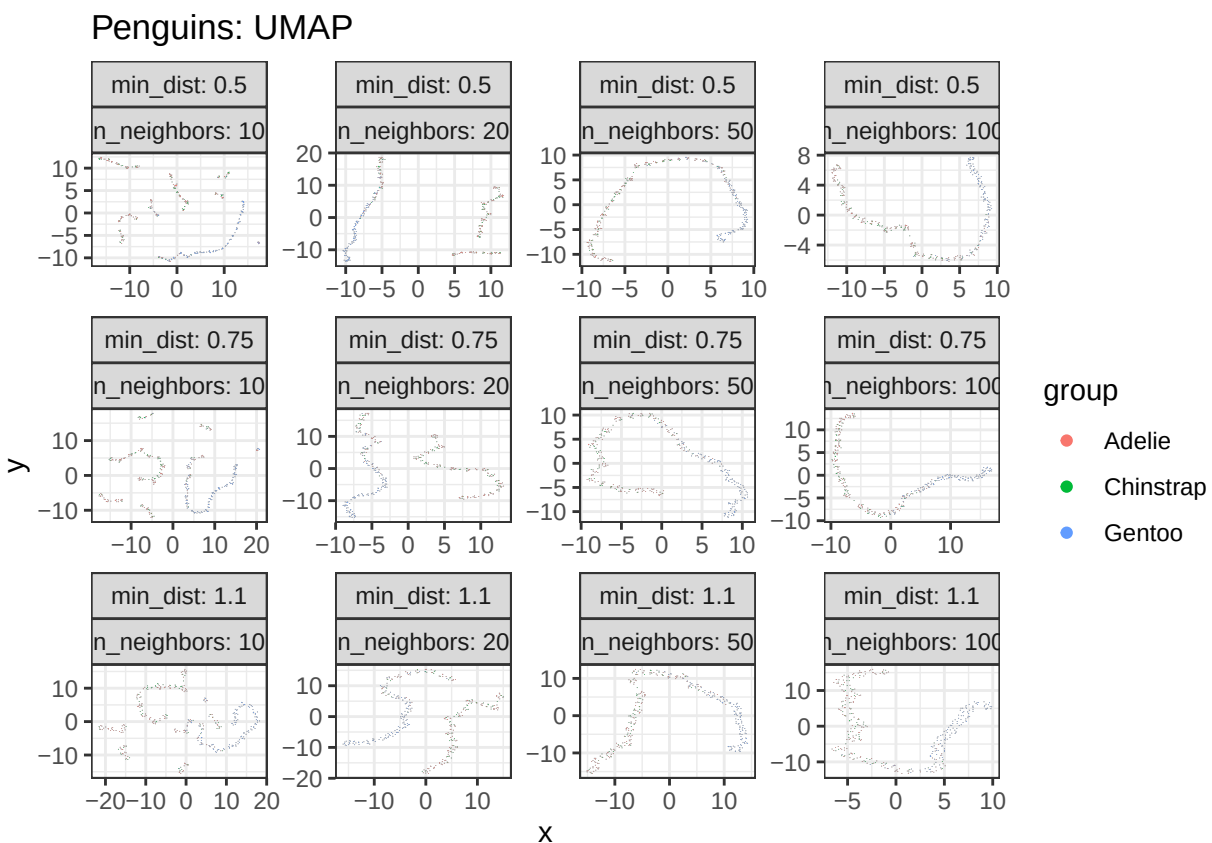
```

    min_dist=umap_params$min_dist[i],
    group=df$species
  )
}))

p <- ggplot(d) +
  geom_scattermore(aes(x=x, y=y, colour=group), pointsize=2) +
  theme(axis.text=element_blank(), axis.ticks=element_blank(),
        axis.title=element_blank(), legend.position="none") +
  facet_wrap(min_dist ~ n_neighbors, labeller=label_both, scales="free") +
  theme_bw() +
  labs(title="Penguins: UMAP")

p

```



## Modelo Final

Nuevamente seleccionamos los parámetros que nos hayan parecido mejores para nuestro modelo final.

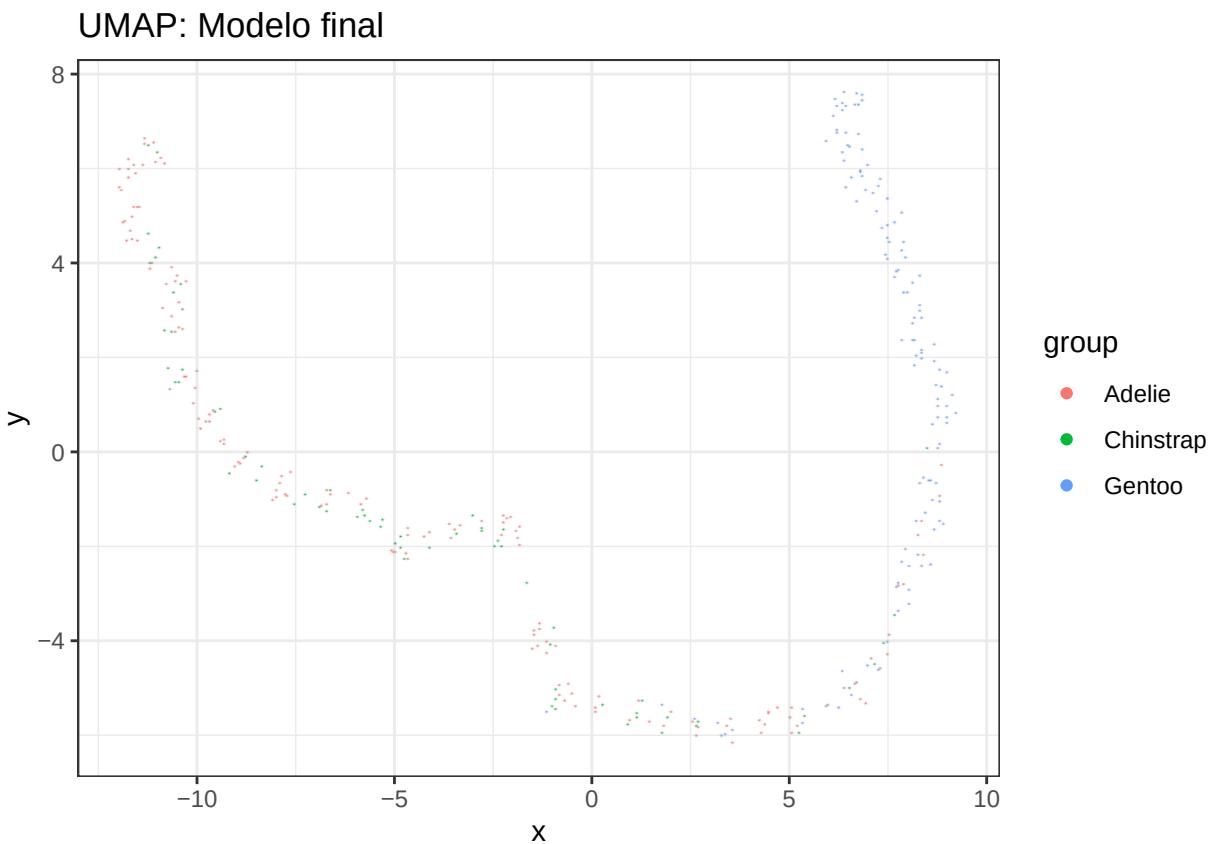
```

Final_plot <- d %>%
  filter(min_dist==0.5, n_neighbors==100) %>%

```

```
ggplot(aes(x=x, y=y, col=group)) +
  geom_scattermore(pointsize=1.1) +
  theme(axis.text=element_blank(), axis.ticks=element_blank(),
        axis.title=element_blank(), legend.position="none") +
  theme_bw() +
  labs(title="UMAP: Modelo final")
```

Final\_plot



## Conclusión

UMAP resulto ser más práctico para trabajar nuestro conjunto de datos, además de que durante la manipulación de la manipulación

se obtuvieron agrupaciones muy interesantes que pueden ayudar a sacar interpretaciones y correlaciones de los datos.